

Ödev 08: Bir Sipariş Sistemi

Küçük bir sipariş sistemi modelleyin, sonra bilerek yarattığınız iki değişmezlik sızıntısını kapatın.

Birinci Bölüm: Money

`Orders.java` dosyasını klasik biçimde oluşturun.

`long cents` ve `String currency` içeren bir `Money` record'u yazın. Kompakt kurucusu şunları yapmalı:

- negatif tutarı reddetsin
- null para birimini reddetsin
- para birimini boşlukları kırparak ve küçük harfe çevirerek **normalleştirsin**
- `gbp`, `eur`, `usd` destekli kümesinin dışındaki her para birimini reddetsin

Sonra `new Money(1250, "gbp")` ile `new Money(1250, "GBP ")` değerlerinin `equals` olduğunu ve ikisini bir `HashSet` içine koymanın tek eleman verdiğini gösterin.

Doğru yapılması gereken bir sıralama ayrıntısı var: desteklenen kümeye karşı denetlemeden **önce** normalleştirin, sonra değil. Nedenini bir yorumda yazın.

Ayrıca farklı para birimlerini toplamayı reddeden `plus(Money)` ve tam kuruşla yüzde hesaplayan `percent(int)` metotlarını ekleyin.

İkinci Bölüm: İki Sızıntı

`String reference` ve `List<String> notes` içeren, kompakt kurucusu **olmayan** bir `LeakyOrder` record'u yazın.

Sonra `main` içinde:

1. Referansını sakladığınız bir listeden bir tane kurun
2. Sonradan *kendi* listenize ekleme yapın ve record'u yazdırın
3. Erişimcisinin döndürdüğü listeye ekleme yapın ve record'u yazdırın

Her iki saldırı da başarılı olacak. Sonra düzeltilmiş sürüm olan `Order` record'unu yazın ve her iki saldırının da başarısız olduğunu gösterin.

Açıklayın

Bir yorumda **her iki** sızıntıyı ayrı ayrı tarif edin. Aynı sızıntı değiller ve birine yapılan düzeltme diğerini otomatik olarak düzeltmez. Sonra Modül 06'ya atıfla, bunun neden sıradan bir sınıftan çok bir record için önemli olduğunu söyleyin.

Üçüncü Bölüm: Kargo ve Durum

Her biri bir açıklama ve **kendi ücret kuralını** sabite özgü davranış olarak taşıyan üç seçenekli bir `Shipping` enum'u yazın. Standart kargo 50.00 üzerinde bedava, altında 3.99. Ekspres her zaman 8.99. Mağazadan teslim her zaman bedava.

Her biri `"PLC"` gibi açık ve **kalıcı bir kod** taşıyan dört sabitli bir `OrderStatus` enum'u yazın. Kodun neden `ordinal()` yerine var olduğunu söyleyen bir yorum ekleyin.

Son olarak, **default** dalı olmayan bir `switch` ifadesiyle duruma göre tavsiye veren bir metot yazın, ve `default` yazmamanın size ne kazandırdığını açıklayan bir yorum ekleyin.

Kabul Kriterleri

- ☐ `java Orders.java` ile çalışıyor
- ☐ `Money` kompakt kurucusunda para birimini normalleştiriyor
- ☐ Bir yorum normalleştir-sonra-doğrula sıralamasını açıklıyor
- ☐ `Money(1250, "gbp")` ile `Money(1250, "GBP ")` eşit, çıktıda gösteriliyor
- ☐ `plus` uyuşmayan para birimlerini reddediyor
- ☐ `LeakyOrder` çıktısı her iki saldırının da başarılı olduğunu gösteriyor
- ☐ `Order` çıktısı her iki saldırının da başarısız olduğunu gösteriyor
- ☐ Bir yorum iki sızıntıyı ayrı ayrı tarif ediyor
- ☐ `Shipping` bir `switch` değil sabite özgü davranış kullanıyor
- ☐ `OrderStatus` nedenini anlatan bir yorumla birlikte açık bir kod taşıyor
- ☐ Tavsiye `switch` ifadesinde `default` yok

İleri Seviye

Tavsiye metodunuza bir `CANCELLED` dalı ekleyin, sonra `OrderStatus` enum'una beşinci bir sabit ekleyin ve `switch`'i **güncellemeyin**. Derleyin, tam hatayı kaydedin ve bir yoruma koyun.

Sonra bir `default` dalı ekleyip yeniden derleyin ve kodun güvenliği açısından neyin değiştiğini anlatın.

İpucu, ikinci bölümün ikinci saldırısı işe yaramıyorsa

Erişimcinizin ne döndürdüğüne bakın. Bir record, alanı saklandığı hâliyle geri veren bir erişimci üretir. Alan düz bir `ArrayList` ise çağırının aldığı şey de odur ve onunla istediğini yapabilir.